

Contents

A small language model that reports its own embedding coordinates	1
Summary	1
1. The question	3
2. Setup	3
3. Measures	4
4. Language modelling	4
5. Accuracy and follow, all runs	5
6. Why the joint runs generalize worse	6
6.1 Rare tokens' embedding vectors	6
6.2 Rescaling test	6
6.3 Interpretation and an open alternative	7
7. Response to the norm of $E[t]$, and LayerNorm	7
8. Editing an embedding vector inside the model	8
9. A linear map from the LM-only model's internal state	9
10. Raising follow	9
10.1 Three changes to training	9
10.2 Continuing the control model	9
10.3 Full measurements	10
10.4 What made the difference	12
10.5 Clean rerun of the slope comparison	12
11. Answers written as text	13
11.1 Design	13
11.2 Runs	13
11.3 Results	13
12. What this does and does not show	14
13. Checks	15

A small language model that reports its own embedding coordinates

Summary

Question. Can a language model learn to report the values of its own weights, with answers computed from the current weights rather than memorized? The weights asked about here are the token embeddings.

Model and task. A small GPT-style transformer (4 blocks, 1.35 million parameters; the internal state at each position, passed from block to block, is a vector in $\mathbb{R}^{128}$) is trained on TinyStories, a corpus of simple children's stories, for 15,000 steps. Input and output embeddings are tied: each of the 4,096 tokens t has one embedding vector $E[t] \in \mathbb{R}^{128}$, used both to read text and to score the next token. Besides predicting the next token, the model is trained on a self-report task: given the three input symbols [QUERY, COORD_ i , t], it must output the number $E[t, i]$, the current value of coordinate i of t 's embedding vector, through a linear output head. Questions are asked about 3,072 randomly chosen tokens (asked-about tokens). The other 1,024 (never-asked tokens) are used only to test whether the answers generalize; all but 47 of them occur in the text the model is trained on.

Runs.

- LM-only (seed 0): next-token prediction only.
- Joint (seeds 0 and 1): next-token prediction plus the self-report task; both tasks update all parameters, including the token embedding vectors.
- Control (seed 0): as joint seed 0, except that the self-report task cannot change the token embedding vectors (its gradient is stopped there). The token embedding vectors then receive

gradient from next-token prediction alone (section 12 notes two indirect effects of the self-report task on them).

- Later runs (sections 10 and 11), all in the control setting: training with perturbed questions (the vector asked about is moved slightly and the target moves with it), a larger loss coefficient, and a loss on the slope of the answers; and a version in which the model writes its answer as text.

Measures.

- Centred R^2 : $1 - \Sigma(\text{answer} - \text{target})^2 / \Sigma(\text{target} - \text{that coordinate's mean over the tokens tested})^2$. It is 1 for exact answers and 0 for always answering the coordinate's mean.
- Follow: J is the 128×128 matrix of derivatives of the 128 answers about t with respect to the 128 coordinates of $E[t]$. Follow is $\text{tr}(J)/128$, averaged over tokens: the average amount by which the answer about coordinate i moves when coordinate i of $E[t]$ moves by one unit. It is 1 if each answer moves exactly with its coordinate, and 0 if the answers do not respond to $E[t]$.
- Other movement: how much the answers move in ways other than following the change of $E[t]$, as a fraction of the size of the change (section 3). 0 for exact answers.

run	validation loss (nats per token)	centred R^2 , asked-about / never-asked tokens	follow, asked-about / never-asked tokens
LM-only, seed 0	2.146		
joint, seed 0	2.248	0.976 / 0.680	0.851 / 0.751
joint, seed 1	2.290	0.955 / 0.504	0.745 / 0.666
control, seed 0	2.290	0.982 / 0.982	0.871 / 0.868
control_slope, seed 0 (section 10)	2.275	0.998 / 0.998	0.997 / 0.998

Findings.

1. The self-report task raises the validation loss by 0.10 nats per token (joint, seed 0) or 0.14 (control), compared with LM-only. There is no LM-only run with seed 1.
2. In the control, answers about never-asked tokens are as accurate as answers about asked-about tokens.
3. In the joint runs, answers about never-asked tokens are much less accurate. The evidence for the cause (seed 0; seed 1 agrees):
 - In LM-only, rare tokens (fewer than 100 occurrences in the training text) end with embedding vectors of norm about 4.7, against 1.4 to 1.7 for the others.
 - In joint seed 0, rare asked-about tokens end with norm 1.53, rare never-asked tokens with 3.95. Answers about the latter respond little to their vectors (follow 0.34) and are inaccurate (rescaling score, defined in section 6.2: -1.34 , where 1 means exact). Rescaled to norm 1.41 (the mean norm of frequent never-asked tokens), keeping their direction, they score 0.96.
 - In the control, rare tokens end with large norms whether asked about or not (4.11 and 4.13), so vectors with large norms are among the questions in training. Answers about rare never-asked tokens score 0.99, although their follow (0.62) is lower than for other tokens (0.90).

Interpretation: when it can, the self-report task changes the asked-about embedding vectors so that the answers work for them; the never-asked vectors keep the shape next-token prediction gave them, and the answers do not carry over to them.

4. In the joint runs and the control, answers are far from exact: other movement is 0.35 to 0.48, and the answers respond less to a change of the norm of $E[t]$ than to changes of its direction (section 7).
5. Follow can be raised close to 1 (section 10). Training from scratch with perturbed questions, loss coefficient $\lambda = 4$ and the slope loss (control_slope) gives follow 0.997 / 0.998, other

movement 0.05 and centred R^2 0.998, with validation loss 2.275 (control: 2.290). Perturbed questions alone, from scratch, gave only 0.885. Which of $\lambda = 4$ and the slope loss is needed is not established; one seed.

6. The model can write its answers as text, with its own output layer: five characters such as “-0.13” (section 11). After training with perturbed questions, the text answers have centred R^2 0.992 / 0.989 and follow 0.96 when $E[t]$ changes by 30% of its norm, against 0.97 for the best number-head continuation (ft_slope) on the same test; every answer is well-formed. Validation loss is 2.193, closer to LM-only (2.146) than any number-head model; why is not known.

1. The question

Can a language model answer questions about its own weights, with answers that come from the weights themselves? The test, as in `../self-report-embeddings/`: change a weight and see whether the answer changes with it. There, a toy model with random embedding vectors and no language modelling learned to answer almost exactly (follow 0.997 to 1.005) once it was asked about 3,072 distinct embedding vectors. Here the embedding vectors are shaped by next-token prediction and shared with the output, and the model has positional embeddings and LayerNorm.

2. Setup

Data. TinyStories (Hugging Face, roneneldan/TinyStories): training shard 0 of 4 (529,930 stories, 122 million tokens; the training text) and the validation set (21,990 stories, 4.9 million tokens). Tokenizer: byte-level BPE with 4,096 tokens, including an end-of-story token, trained on the training text.

Model. A GPT-style decoder: 4 blocks (each: LayerNorm, causal self-attention with 4 heads, LayerNorm, feed-forward layer of width 512 with GELU), internal states in $\mathbb{R}^{128}$, learned positional vectors $P[0], \dots, P[127]$, context of 128 tokens. Each block applies LayerNorm to its input before using it: subtract the mean of the 128 coordinates, divide by their standard deviation, then apply a learned scale and shift per coordinate. 1,350,657 parameters in all.

Embedding table. One vector in $\mathbb{R}^{128}$ for each of the 4,096 tokens and for each of 129 extra symbols (QUERY, COORD_0, ..., COORD_127). The input to the first block at a position is the embedding vector of the symbol there plus the positional vector of that position. At the output, the score of each possible next token is the inner product of the last internal state (after a final LayerNorm) with that token’s embedding vector (tied embeddings). The extra symbols are never predicted.

Self-report task. A question (t, i) is the sequence [QUERY, COORD_i, t]; t is at position 2 (counting from 0). The answer is $w \cdot h + b$, where h is the internal state at position 2 after the last block and before the final LayerNorm, and $w \in \mathbb{R}^{128}$ and b are the weights of the output head. The target is $E[t, i]$, the current value. The task loss on a batch of 256 questions is $1 - R^2$ of the batch, where this R^2 uses a single mean over the batch (uncentred R^2 , unlike the centred R^2 used for measurement). No gradient flows through the targets.

Asked-about and never-asked tokens. A random three quarters of the 4,096 tokens (3,072) are asked about during training; the other 1,024 are never asked about. The choice depends on the seed. Both kinds occur in the training text, except 88 asked-about and 47 never-asked tokens (seed 0) that never occur in it.

Training, all runs. 15,000 steps of AdamW (learning rate $2 \cdot 10^{-3}$ after 200 warm-up steps, cosine decay to $2 \cdot 10^{-4}$; weight decay 0.1 on the block and output-head matrices only; the norm of the combined gradient clipped at 1). Each step uses 32 windows of 128 tokens from the training text (61 million tokens seen in all), and, in the joint runs and the control, 256 random questions about asked-about tokens. Loss: next-token cross-entropy plus λ times the self-report loss, with loss coefficient $\lambda = 1$.

The four runs.

run	self-report task	gradient of the self-report loss reaches the token embedding vectors	seed
LM-only	no	(no self-report loss)	0
joint	yes	yes	0 and 1
control	yes	no (stopped at $E[t]$; it still trains the blocks, the output head, and the QUERY and COORD vectors)	0

Runs with the same seed start from the same weights and see the same text in the same order. Each run took 2.0 to 2.3 hours on 2 CPU threads.

3. Measures

All measures are taken after training, with all weights fixed.

- Validation loss: average of $-\log(\text{probability given to the actual next token})$ over 100×32 fixed windows of 128 tokens of the validation set, in nats per token. (Section 4 also uses the training logs, which used 20×32 windows.)
- Centred R^2 (defined in the summary), over all 128 questions about every token of the set measured.
- J , follow (defined in the summary). For a unit vector $u \in \mathbb{R}^{128}$, the gain along u is $u^T J u$: how much the answers move along u , per unit change of $E[t]$ along u . Follow is the average gain over the 128 coordinate axes.
- Other movement: for each token, $\|J - fI\|_F / \sqrt{128}$, where f is that token’s follow; averaged over tokens. It is the root-mean-square size, over random directions of a small change δ , of the part of the answers’ movement that is not $f\delta$, as a fraction of $|\delta|$. 0 for exact answers.
- Finite-change follow: add to $E[t]$ a vector δ in a random direction with norm 10% of $|E[t]|$; if the 128 answers change by Δ , the value is $\Delta \cdot \delta / |\delta|^2$.
- Token frequency: number of occurrences in the training text. Rare: fewer than 100; common: 100 to 9,999; frequent: 10,000 or more.

Follow, other movement and the gains use 512 random asked-about and 512 random never-asked tokens, unless stated otherwise.

4. Language modelling

Validation loss: LM-only 2.146; joint seed 0 2.248 (+0.102); joint seed 1 2.290; control 2.290 (+0.144).

Joint seed 0 against LM-only during training (training-log windows):

step	joint, seed 0	LM-only	difference
1,500	3.094	2.854	0.240
4,500	2.609	2.487	0.122
9,000	2.388	2.284	0.104
15,000	2.261	2.154	0.107

The difference shrinks during the first half of training and stays between 0.10 and 0.11 from step 7,500 on. LM-only reached joint seed 0’s final loss on these windows (2.261) at about step 9,700.

This difference may come from the two tasks competing for model capacity, or from optimization: the two losses share Adam’s running statistics and one gradient-norm clip. This experiment does not tell the two causes apart.

Samples (prompt “Once upon a time”, temperature 0.8, three per run, in `results/measure_<run>.json`) are fluent children’s-story text in every run, with lapses in coherence in every run, LM-only included. For example, joint seed 0: “Once upon a time, there was a little girl named Lily. She loved to play with her toys and her toys. One day, she went to the park with her mommy. She saw the big red ball that was round and shiny.” and “The knight was always not sure to turn and be successful.”

5. Accuracy and follow, all runs

measure	joint, seed 0	joint, seed 1	control, seed 0
centred R^2 , asked-about tokens	0.976	0.955	0.982
centred R^2 , never-asked tokens	0.680	0.504	0.982
centred R^2 , random vectors	0.813	0.699	0.821
follow, asked-about tokens (std over tokens)	0.851 (0.02)	0.745 (0.03)	0.871 (0.11)
follow, never-asked tokens (std over tokens)	0.751 (0.16)	0.666 (0.15)	0.868 (0.11)
finite-change follow, mean: asked-about / never-asked	0.848 / 0.748	0.742 / 0.661	0.866 / 0.861
other movement: asked-about / never-asked	0.41 / 0.40	0.48 / 0.46	0.35 / 0.35

Random vectors: 1,024 vectors drawn from a normal distribution with the mean and standard deviation of each coordinate over all 4,096 embedding vectors, put at position 2 in place of $E[t]$; question i asks for their coordinate i . LM-only is not in the table: its output head was never trained (its follow is -0.001).

Observations.

- In the control, the answers are equally accurate for asked-about and never-asked tokens (0.982 and 0.982), and equally responsive (follow 0.871 and 0.868).
- In the joint runs, the answers are much less accurate for never-asked tokens than for asked-about tokens (0.680 against 0.976; 0.504 against 0.955), and less responsive.
- In all runs, follow is well below 1 and other movement is 0.35 to 0.48: the answers move partly with the coordinate asked about and partly in other ways. For comparison, the toy model of `../self-report-embeddings/` reached follow 0.997 to 1.005 and other movement 0.06 to 0.10.
- The two joint seeds differ a lot (0.680 and 0.504); both are far below the control (0.982). There is one control run, so the control’s own variation between seeds is unknown.

6. Why the joint runs generalize worse

6.1 Rare tokens' embedding vectors

Mean norm of $E[t]$, by token frequency (rare / common / frequent):

run	asked-about tokens	never-asked tokens
LM-only, seed 0 (seed-0 split)	4.68 / 1.65 / 1.44	4.77 / 1.68 / 1.46
joint, seed 0	1.53 / 1.44 / 1.33	3.95 / 1.75 / 1.41
joint, seed 1	1.66 / 1.50 / 1.37	4.33 / 1.80 / 1.42
control, seed 0	4.11 / 1.70 / 1.41	4.13 / 1.75 / 1.43

Follow by token frequency (rare / common / frequent; at most 64 tokens per group):

run	asked-about tokens	never-asked tokens
joint, seed 0	0.85 / 0.85 / 0.85	0.34 / 0.79 / 0.83
joint, seed 1	0.73 / 0.75 / 0.74	0.27 / 0.68 / 0.73
control, seed 0	0.62 / 0.90 / 0.90	0.62 / 0.90 / 0.90

Seed 0 has 248 rare asked-about and 97 rare never-asked tokens. The rare embedding vectors are nearly parallel (mean pairwise cosine 0.998 for the rare never-asked tokens of joint seed 0 and of the control), so they test essentially one direction, not dozens.

Observations.

- Next-token prediction alone gives rare tokens' vectors about three times the norm of the others' (LM-only).
- In the joint runs, rare asked-about tokens end with ordinary norms; rare never-asked tokens end with large norms; and follow on the rare never-asked tokens is low (0.34, 0.27).
- In the control, rare tokens end with large norms in both groups, and follow is the same for both groups (0.62).

6.2 Rescaling test

Each vector is rescaled to a given norm, keeping its direction, and the answers about it are scored by the rescaling score: $1 - (\text{mean over the vectors of the squared distance between the 128 answers and the vector}) / (\text{mean squared distance of all 1,024 never-asked vectors from their coordinate means})$.

never-asked vectors	joint, seed 0	joint, seed 1	control, seed 0
rare, as they are	-1.34	-3.47	0.99
rare, rescaled to the norm of the frequent ones	0.96	0.90	0.97
frequent, as they are	0.94	0.91	0.97
frequent, rescaled to the norm of the rare ones	-1.67	-2.75	-1.45

Observations.

- In the joint runs, the rare never-asked vectors are answered badly at their own norm and well at the frequent tokens' norm.

- In the control, the rare never-asked vectors are answered well at their own norm.
- In all three runs, frequent vectors rescaled to the rare tokens' norm are answered badly.

So whether a vector with a large norm is answered well depends on whether asked-about vectors of similar norm and direction existed in training. In the control, the rare vectors (norms about 4, all nearly in one direction) include asked-about tokens: 348 asked-about vectors have norm above 3. In the joint runs, no asked-about vector has norm above 1.87 (seed 0: 1.83), while never-asked vectors reach 4.06 (seed 0) and 4.45 (seed 1). Frequent vectors rescaled to norm about 4 along their own directions are unlike anything asked about in any run.

6.3 Interpretation and an open alternative

Interpretation. In the joint runs, the self-report task changes the embedding vectors of the asked-about tokens: among other changes, it keeps the norms of the rare ones small. The answers are then fitted to asked-about vectors whose distribution differs from that of the never-asked vectors, and they do not carry over. In the control, asked-about and never-asked vectors come from the same distribution, and the answers carry over completely. Letting the self-report task change the embedding vectors costs less next-token loss than forbidding it: +0.10 nats per token (joint, seed 0) against +0.14 (control), one run each.

What the control establishes: the gradient of the self-report loss on the token embedding vectors is what keeps the norms of the rare asked-about vectors small. What it does not establish is how. Two mechanisms fit:

1. The gradient moves those vectors toward a region where the answers are accurate.
2. An effect of Adam: Adam divides each parameter's step by a running average of the size of its recent gradients. A rarely used embedding vector with a small but consistent gradient from next-token prediction therefore takes full-size steps and its norm grows. An additional, noisy gradient on the same vector (here from the self-report task) enlarges that average and slows the growth, whatever its direction.

7. Response to the norm of $E[t]$, and LayerNorm

The norm of $E[t]$ can change along the direction of $E[t]$ itself. Gains $(u^T J u)$ along that direction and, on average, along the 127 directions perpendicular to it:

run	gain along $E[t]$: asked-about / never-asked	gain perpendicular to $E[t]$: asked-about / never-asked
joint, seed 0	0.66 / 0.49	0.85 / 0.75
joint, seed 1	0.57 / 0.42	0.75 / 0.67
control, seed 0	0.75 / 0.75	0.87 / 0.87

In every run, the answers respond less to a change of the norm of $E[t]$ than to a change of its direction.

An architectural fact explains part of this. The input of the first block at position 2 is $v = E[t] + P[2]$. Its LayerNorm first subtracts the mean of v 's coordinates, giving the centred vector c , and then divides by the standard deviation, which is proportional to $|c|$. Multiplying c by a positive number therefore changes nothing that the first block computes. So a change of $E[t]$ along $u = c/|c|$ is invisible to the first block; later blocks receive it only through the internal state, which carries it forward unchanged. $P[2]$ has norm 1.1 to 1.4, comparable to $|E[t]|$, so the direction of $E[t]$ and u are not the same, but they overlap (cosine 0.62 to 0.77). Measured gains along u (64 tokens per group):

run	gain along u: asked-about / never-asked	cosine between E[t] and u: asked-about / never-asked
joint, seed 0	0.005 / 0.006	0.62 / 0.68
joint, seed 1	0.022 / 0.015	0.72 / 0.77
control, seed 0	0.24 / 0.24	0.69 / 0.68

Observation. The joint models barely respond along u (0.005 to 0.022); the control responds partly (0.24). The near-insensitivity along u is therefore not forced by the architecture; it is what training produced in the joint runs. The overlap of u with the direction of E[t] is consistent with the weaker response to the norm of E[t].

A second, exact fact, used as a check of the code: every LayerNorm subtracts the mean, so a change of E[t] along the all-ones direction (1, ..., 1) reaches the answers only through the output head, which uses the internal state directly, and moves all 128 answers by the same amount, Σw . Measured: all 128 entries of J·1 equal Σw (0.7466 for joint seed 0), with a standard deviation over the 128 answers of about $1.2 \cdot 10^{-7}$, which is single-precision rounding.

8. Editing an embedding vector inside the model

Purpose: to confirm that changing a stored embedding vector changes both the answers about that token and the model's use of the token in text, and to measure by how much. Some response of both kinds is guaranteed, since the edited vector is both an input and an output weight; the measurement is of size.

For each of the 20 never-asked tokens that occur most often in 20×32 validation windows (among them " it", " he", " in", " said", " she", " his", " day"), I added to its stored embedding vector a change of 10% of its norm in a random direction, and compared the model's outputs with and without the change.

	joint, seed 0	joint, seed 1	control, seed 0
finite-change follow of the answers about the edited token: range over the 20 tokens (mean)	0.73 to 0.90 (0.82)	0.61 to 0.90 (0.72)	0.72 to 0.91 (0.83)
change of the next-token distribution where the edited token is the current input, divided by the change at positions with no occurrence of the token so far (ratio of the means over the 20 tokens)	32	34	31
the same, with all positions of all 20 tokens pooled	27	30	28
tokens for which the first change is larger	19 of 20	20 of 20	19 of 20

The change of the next-token distribution is the Kullback–Leibler divergence of the edited from

the original distribution. It is not zero at positions without the token, because the edited vector is also used to score that token as a possible next token at every position.

9. A linear map from the LM-only model's internal state

To see how much information about $E[t]$ the LM-only model's internal state already holds: a linear map from the internal state of the LM-only model at position 2 (input [QUERY, COORD_0, t]; the QUERY and COORD vectors of this model are untrained) to $E[t]$, fitted by ridge regression on the asked-about tokens, gives centred R^2 0.834 on the never-asked tokens (seed-0 split; 0.803 with the seed-1 split). This map has a separate weight vector for each of the 128 coordinates; the models' output head is a single weight vector, and the coordinate asked about enters only through the COORD_i symbol. Takeaway: a simple readout of $E[t]$ from an LM-only model's internal state generalizes to never-asked tokens better than the joint models' answers (0.680, 0.504) and worse than the control's (0.982).

10. Raising follow

The control's follow is 0.871 / 0.868 (asked-about / never-asked tokens). The aim of this section is to raise it toward 1 without losing accuracy or generalization to never-asked tokens. All runs here keep the control setting: the self-report gradient is stopped at the token embedding vectors.

10.1 Three changes to training

Perturbed questions. A question (t, i) normally puts $E[t]$ at position 2 and asks for $E[t, i]$. A perturbed question puts $x = E[t] + \delta$ there instead, where δ points in a random direction and has norm $s \cdot |E[t]|$, with s drawn uniformly from [0, 0.5]; the target is x_i . The model then sees questions near each embedding vector, not only at it. Follow measures exactly this response to nearby inputs, so these questions train for it directly.

Larger loss coefficient. $\lambda = 4$ instead of 1.

Slope loss. For a question (t, i), draw a δ in a random direction with norm $0.05 \cdot |E[t]|$, and compare the answer about coordinate i at $E[t] + \delta$ with the answer at $E[t]$. Their difference should be δ_i . The loss is the mean of $(\text{difference} - \delta_i)^2$, divided by the mean of δ_i^2 , over the batch; it is added to the training loss with coefficient 1. For a small δ the difference is approximately $(J\delta)_i$, so this loss pushes J toward the identity along random directions.

10.2 Continuing the control model

Each run below continues training from a finished model for a further number of steps, with a fresh optimizer at a constant learning rate of $2 \cdot 10^{-4}$ (the control's final learning rate). The four runs that start from the control see text windows and questions that the control did not see. A code review found that the two runs that start from ft_lam4_jit did not get new data: because of a seeding error, ft2_lam4_jit's first 3,000 steps repeat ft_lam4_jit's text windows and questions exactly, and ft_slope repeats its text windows (its questions differ after the first step, because the slope loss draws extra random numbers). Measurements are not affected (they use their own tokens and directions), but the comparison of ft2_lam4_jit with ft_slope is confounded by this; a clean rerun of that comparison is in section 10.5. Measured on 128 asked-about and 128 never-asked tokens (the same tokens for every run), with quick_measure.py:

run	starts from	steps	λ	perturbed questions	slope loss	follow: asked-about / never-asked	other movement	centred R^2	validation loss (training-log windows)
(control)						0.870 / 0.854	0.35	0.982	2.307
ft_more	control	3,000	1	no	no	0.893 / 0.878	0.33	0.986	2.295
ft_lam4	control	3,000	4	no	no	0.896 / 0.880	0.31	0.987	2.302
ft_jit	control	3,000	1	yes	no	0.959 / 0.949	0.28	0.980	2.298
ft_lam4_jit	control	3,000	4	yes	no	0.968 / 0.959	0.27	0.979	2.307
ft2_lam4_jit	ft_lam4_jit	6,000	4	yes	no	0.980 / 0.973	0.18	0.989	2.290
ft_slope	ft_lam4_jit	3,000	4	yes	yes	0.994 / 0.988	0.14	0.990	2.304

Observations.

- More training and a larger λ raise follow only a little (to about 0.89).
- Perturbed questions raise it to 0.96 to 0.97 within 3,000 steps.
- Continuing ft_lam4_jit for 6,000 more steps adds about 0.01. Follow moves by about ± 0.02 from one evaluation to the next (every 1,000 steps), so single evaluations of these runs differ by more than the differences among the last rows of this table; the 6,000-step trend is small.
- The slope loss, started from the same model with otherwise the same settings (but see the data caveat above), gives 0.994 / 0.988 after 3,000 steps and lowers other movement to 0.14, the lowest of all runs here.
- Centred R^2 does not fall, and the validation loss stays within 2.290 to 2.307.

10.3 Full measurements

The same measures as in section 5, on 512 asked-about and 512 never-asked tokens, for the control, the two best continuations, and two runs trained from scratch for 15,000 steps exactly like the control except as stated:

- control_jit: perturbed questions ($\lambda = 1$);
- control_slope: perturbed questions, $\lambda = 4$, and the slope loss, the settings of ft_slope's last 3,000 steps.

measure	control	control_jit (from scratch)	ft2_lam4_jit	ft_slope	control_slope (from scratch)
follow, asked-about / never-asked	0.871 / 0.868	0.885 / 0.883	0.979 / 0.977	0.994 / 0.992	0.997 / 0.998
standard deviation of follow over tokens	0.11	0.04	0.03	0.03	0.01

measure	control	control_jit (from scratch)	ft2_lam4_jit	ft_slope	control_slope (from scratch)
finite-change follow (10% change), mean	0.866 / 0.861	0.885 / 0.882	0.976 / 0.973	0.991 / 0.989	0.997 / 0.997
other movement	0.35	0.34	0.18	0.14	0.05
gain along E[t] (section 7)	0.75 / 0.75	0.84 / 0.84	0.92 / 0.92	0.93 / 0.92	0.97 / 0.97
gain along u, the direction the first block cannot see (section 7)	0.24	0.22	0.56	0.66	0.93
follow by frequency: rare / common / frequent (asked-about)	0.624 / 0.899 / 0.901	0.815 / 0.899 / 0.886	0.943 / 0.990 / 0.976	0.978 / 1.003 / 0.985	0.991 / 1.000 / 0.995
centred R ² , asked-about / never-asked	0.982 / 0.982	0.970 / 0.970	0.989 / 0.989	0.990 / 0.990	0.998 / 0.998
centred R ² , random vectors (section 5)	0.821	0.852	0.955	0.967	0.995
rescaling score, frequent vectors grown to the rare tokens' norm (section 6.2)	-1.45	-0.26	0.20	0.24	0.81
edit test (section 8): finite-change follow of the edited tokens, range (mean)	0.72 to 0.91 (0.83)	0.73 to 0.94 (0.84)	0.88 to 1.01 (0.93)	0.86 to 1.01 (0.93)	0.96 to 1.01 (0.98)
validation loss (100 × 32 windows)	2.290	2.298	2.274	2.289	2.275

Observations.

- control_slope, trained from scratch, is the best model on every measure in the table: follow 0.997 / 0.998, other movement 0.05, centred R² 0.998. Its other movement is in the range of the toy model of ../self-report-embeddings/ (0.06 to 0.10).
- Follow is close to 1 for rare, common and frequent tokens alike (0.99 to 1.00), and for asked-about and never-asked tokens alike.

- Its answers are also accurate on inputs that are not embedding vectors: random vectors (centred R^2 0.995) and frequent tokens' vectors grown to the rare tokens' norm (rescaling score 0.81, where 1 means exact; the control scores -1.45).
- The response to the norm of $E[t]$ is now close to the response to its direction (gain along $E[t]$ 0.97), and the gain along u is 0.93. So the architectural fact of section 7, that the first block cannot see changes along u , does not prevent the model as a whole from responding along u .
- In the continuations, the edited tokens of section 8 (the 20 most frequent never-asked tokens) have lower follow (mean 0.93) than frequent tokens in general (0.98); in control_slope they do not (0.98). I have not looked into why.
- Validation loss is no worse than the control's (2.275 against 2.290), so the added training costs nothing measurable in next-token prediction; the cost relative to LM-only is 0.13 nats per token.

10.4 What made the difference

control_jit, trained from scratch with perturbed questions alone ($\lambda = 1$), gained little (follow 0.885 against 0.871), and only for rare tokens (0.82 against 0.62). Continuing it for 3,000 steps exactly like ft_jit ($\lambda = 1$, perturbed questions) raised follow from 0.886 to 0.929 (128 + 128 tokens), while the same continuation of the control raised it from 0.870 to 0.959. I do not know why perturbed questions alone did so much better as a continuation.

control_slope differs from control_jit in two settings at once, $\lambda = 4$ and the slope loss, so these runs do not say which of the two matters, or whether both are needed. In the continuations, $\lambda = 4$ without perturbed questions added little (ft_lam4: 0.896 against 0.893 for ft_more). Perturbed questions gave the largest single gain (ft_jit: 0.959). Added on top of perturbed questions, the slope loss gave a further gain, smaller with new data (section 10.5) but present on every measure. One run per setting, seed 0.

10.5 Clean rerun of the slope comparison

Two continuations of ft_lam4_jit, 3,000 steps each, $\lambda = 4$, perturbed questions, both with text windows and questions that no earlier run saw (the same text windows for both; the questions differ after the first step, because the slope loss draws extra random numbers): fresh_noslope without the slope loss, fresh_slope with it. Measured on 512 + 512 tokens:

measure	fresh_noslope	fresh_slope
follow, asked-about / never-asked	0.975 / 0.973	0.987 / 0.986
finite-change follow (10% change), mean	0.972 / 0.969	0.985 / 0.983
other movement	0.224	0.135
gain along $E[t]$	0.89 / 0.89	0.94 / 0.94
centred R^2	0.987 / 0.987	0.992 / 0.992
validation loss	2.287	2.284

With new data, the slope loss still helps on every measure: follow rises by 0.012, and other movement falls from 0.22 to 0.13. The difference in follow is smaller than in the confounded comparison of section 10.2 (0.994 against 0.980, on 128 + 128 tokens). One run each.

11. Answers written as text

11.1 Design

In sections 2 to 10 the answer is a number produced by a separate output head. Here the model writes the answer with its ordinary output layer, as text.

Format. The answer to question (t, i) is $E[t, i]$ rounded to two decimals and written with five single-character tokens: a sign ('+' or '-'), a digit, '.', and two digits; for example -0.127 is written `"-0.13"`. Values are clipped to ± 9.99 (no coordinate of these models comes close). All five characters are single tokens of the tokenizer (checked: the tokenizer encodes `"-0.13"` as the five tokens `'-', '0', '.', '1', '3'`).

Sequence and loss. The full sequence is $[QUERY, COORD_i, t, a_0, a_1, a_2, a_3, a_4]$, where $a_0 \dots a_4$ are the answer characters. The model predicts each answer character from the symbols before it, exactly as it predicts the next token of a story: the output at position 2 predicts a_0 , the output at position 3 predicts a_1 , and so on. The self-report loss is the cross-entropy of the five answer characters, averaged. $\lambda = 1$ unless stated.

Reading an answer. Greedy decoding: at each of the five positions take the most likely token, restricted to the characters allowed at that position (sign, digit, '.', digit, digit). Section 11.3 also measures how often the unrestricted most likely tokens form a valid answer.

Control setting. As in the control of section 2, the self-report task must not change the token embedding vectors it is asked about. With tied embeddings there are two paths by which it could: the input $E[t]$ at position 2, and the output layer, which scores every token by its embedding vector, so the cross-entropy reaches every token's vector. The gradient is stopped at the input, and in the output layer only the vectors of the 13 answer characters (sign, digits, '.') receive gradient from the self-report loss. Those 13 tokens are never asked about and are left out of all measurements. A test checks that the self-report loss gives no gradient to any other token's embedding vector.

Follow for text answers. The answers move in steps of 0.01. In the text model, the median norm of a token's embedding vector is 1.56, so its coordinates have root-mean-square size 0.14; a change of 10% of $|E[t]|$ moves each coordinate by about 0.014 (root mean square), close to the rounding step. Follow is therefore measured with finite changes of 30% of $|E[t]|$ (section 3 used 10%), over 256 asked-about and 256 never-asked tokens. The same tokens and the same random changes are given to the number-head models, for a paired comparison. The Jacobian measures of section 3 do not apply, because the text answers are not differentiable.

11.2 Runs

- text: trained from scratch like the control (15,000 steps, seed 0, the same text windows and the same schedule), with the text answer in place of the number head.
- Continuations of it, as in section 10 (fresh optimizer, constant learning rate $2 \cdot 10^{-4}$), all with perturbed questions; the target is the perturbed vector's coordinate, written as text. `text_jit`: 3,000 steps, $\lambda = 1$. Then `text_jit_lam4`: 3,000 steps, $\lambda = 4$. Then `text_jit_lam4_long`: 6,000 steps, $\lambda = 4$. Then `text_jit_lam4_long2`: 6,000 steps, $\lambda = 4$, with new text windows and questions (after the seeding error below was fixed). The slope loss of section 10 needs differentiable answers and was not used.
- Because of the seeding error described in section 10.2, `text_jit_lam4` repeats `text_jit`'s 3,000 batches (text windows, questions and perturbations), and `text_jit_lam4_long` repeats them once more in its first 3,000 steps. The 12,000 continuation steps therefore contain 6,000 distinct batches.

11.3 Results

Measured with `measure_text.py`. Centred R^2 is over all asked-about and all never-asked tokens. Follow is finite-change follow with changes of 30% of $|E[t]|$, as described in 11.1; every model here

gets the same 256 + 256 tokens and the same changes. Valid-format rate: the fraction of answers whose unrestricted most likely tokens form a valid answer, over 64 tokens $\times$ 128 coordinates per set.

model	centred R^2 : asked-about / never-asked	follow (30% change), mean: asked-about / never-asked	valid-format rate	validation loss (100 $\times$ 32 windows)
text	0.951 / 0.948	0.756 / 0.739	1.00 / 1.00	2.180
text_jit_lam4 (+ 6,000 steps)	0.975 / 0.973	0.862 / 0.855	1.00 / 1.00	2.180
text_jit_lam4_long (+ 12,000 steps)	0.989 / 0.986	0.943 / 0.942	1.00 / 1.00	2.187
text_jit_lam4_long2 (+ 18,000 steps)	0.992 / 0.989	0.965 / 0.961	1.00 / 1.00	2.193
number head: control	0.982 / 0.982	0.841 / 0.837		2.290
number head: ft_slope	0.990 / 0.990	0.970 / 0.972		2.289
LM-only				2.146

Observations.

- The text model writes a well-formed answer every time, even without the restriction to allowed characters.
- Trained from scratch like the control, the text answers are less accurate than the control's number answers (0.951 against 0.982) and follow less (0.756 against 0.841).
- Continuing with perturbed questions raises both. After 18,000 more steps the text answers are as accurate as the best number-head model (centred R^2 0.992 / 0.989 against 0.990 / 0.990) and follow almost as well: 0.965 / 0.961 against 0.970 / 0.972 for ft_slope, on the same tokens and changes.
- At this size of change ft_slope's follow is 0.97, lower than its 0.99 at 10%: its answers respond slightly less to large changes than to small ones.
- During the last continuation, follow on the 64 + 64 tokens checked every 1,000 steps varied between 0.94 and 0.99 (asked-about tokens) without a clear trend, so these 6,000 steps may have added little; the full measurement (256 + 256 tokens) rose from 0.943 to 0.965.
- The text model's validation loss is 2.180 to 2.193, against 2.290 for the number-head control and 2.146 for LM-only. So the self-report task, written as text, costs the language model 0.03 to 0.05 nats per token, against 0.14 with the number head. I have not found the reason. One difference: the number head's loss ($1 - R^2$ of the batch) and the text loss (cross-entropy) have different sizes and gradients, so $\lambda = 1$ is not the same weight in the two cases. Another: the number head reads the internal state before the final LayerNorm and must express $E[t]$ linearly there, while the text answer goes through the same final LayerNorm and output layer as next-token prediction. Neither has been tested.
- Stories from the text models are fluent children's-story text with lapses in coherence, like those of every other run (three per model in results/measure_<model>.json).

12. What this does and does not show

- A small language model can be trained to answer questions about its own embedding coordinates with answers that respond to the current embedding vector and are as accurate for tokens never asked about as for tokens asked about. In the control, follow is 0.87 and centred R^2 0.982, at a cost of 0.14 nats per token in next-token prediction.
- Training with perturbed questions, $\lambda = 4$ and the slope loss, from scratch, gives follow 0.997 / 0.998, other movement 0.05 and centred R^2 0.998, for asked-about and never-asked tokens

alike, at the same cost in next-token prediction as the control (section 10). The answers then respond almost equally to changes of the norm and of the direction of $E[t]$ (gain along $E[t]$ 0.97), and are accurate on random vectors too (centred R^2 0.995). Which of the three changes are needed is not established (section 10.4).

- Perturbed questions alone, from scratch, helped little (follow 0.885); the same training helped much more as a continuation of a finished model. Why is not known.
- The answers can be written as text, by the model's own output layer, in a fixed five-character format. After continued training with perturbed questions they are about as accurate as the best number answers (centred R^2 0.992 / 0.989) and follow 0.96 at a 30% change, close to the best number answers on the same test (0.97; section 11). This version cost the language model much less (0.03 to 0.05 nats per token); why is not known.
- If the self-report task may change the embedding vectors it is asked about, it does, and its answers then generalize much worse to the other tokens.
- In the control setting the self-report gradient never reaches the token embedding vectors, but the self-report task still affects how they are trained, in two indirect ways: it trains the blocks through which the next-token gradient reaches the embedding vectors, and the gradient norm is clipped for the sum of both losses, so the self-report loss changes the size of each update (for one batch, the clip factor is 0.59 for the control's combined loss against 0.89 for its next-token loss alone, and about 0.05 against 0.67 for the last text model, whose self-report gradient is much larger). "Shaped by next-token prediction alone" in the summary should be read with this qualification.
- The embedding vector of t is the input at t 's position, so the model is reporting a value present in its own input. Weights that are not inputs (for example the feed-forward matrices) remain untested (PLAN.md, roadmap).
- Not tested: training only the self-report path on top of a fixed LM-only model; other model sizes; more seeds (one run per setting, except the joint runs).

13. Checks

- A separate Claude instance reviewed the code before the runs, another a draft of this report, and a third the terms used in it; their findings were checked and the fixes are recorded in NOTES.md.
- 22 tests (tests/test_lm.py), including checks of every measure on hand-made functions with known answers, the all-ones fact of section 7, and that the control's self-report losses (number head, perturbed questions, text answers) give no gradient to the token embedding vectors of text tokens other than the 13 answer characters. The slope loss is tested on an exact reader (loss 0) and a constant one (loss 1); the text format on encoding, decoding, the tokenizer, and teacher-forcing positions.
- Before the text continuations, a 2-step run of finetune_text.py was checked against the numbers of the model it starts from (they agreed).
- Scripts: train.py; measure.py (sections 5 and 8; files results/measure_json); analyze_review.py (follow by frequency in 6.1, 6.2, gains along u in 7, section 9; files results/review_checks_json); norms_by_frequency.py (norm tables in 6.1; results/norms_by_frequency.json); summarize.py. Section 10: finetune.py and quick_measure.py (logs in results/json). Section 11: textanswer.py, train_text.py, finetune_text.py, measure_text.py (results/measure_json).